

Relatório de Prática: Assembly MIPS

Nome: Vítor Albuquerque Dinelli

Disciplina: Laboratório de Arquitetura e Organização de Computadores I

Professora: Poliana Corrêa

Data: 27/08/2026

Objetivo

Apresentar a codificação, decisões arquiteturais e funcionamento de programas escritos em Assembly MIPS utilizando instruções aritméticas, de transferência de dados e desvios condicionais, baseados nos exercícios propostos para entrega na Prática 1.

Exercício 1: Somatório

Lógica da Solução e Análise de Execução

O algoritmo do somatório foi projetado para economizar recursos, utilizando o próprio registrador do valor de entrada (\$s0) para acumular o resultado final. O controle da repetição é feito com \$s1, que inicia com o valor de \$s0 menos um e é decrementado a cada ciclo. A condição de continuidade do loop baseia-se na verificação se \$s1 é maior que zero, implementada pela combinação das instruções slt e beq. Durante as simulações, inserindo o número 5, o programa processou as somas corretamente, resultando no valor 15 (armazenado como 0x0000000f em \$s0). O fim do laço ocorre exatamente quando o contador \$s1 atinge zero, o que valida a lógica de controle da repetição.

Mapeamento de Registradores

Registrador	Função
\$v0, \$a0	Utilizados para chamadas de sistema (syscall) de leitura e impressão na tela.
\$s0	Armazena o valor lido e atua como acumulador do somatório.

\$s1	Contador do laço e próximo valor a ser somado.
\$t0	Sinalizador (flag) para avaliar a condição do desvio.

Exercício 2: Sequência Par Limitada

Lógica da Solução e Análise de Execução

Para gerar a sequência solicitada, a estratégia adotada foi imprimir o número 1 antes de entrar no laço principal, pois todos os demais elementos seguem uma lógica estrita de números pares crescentes. O limite de termos 'N' fornecido pelo usuário fica alocado em \$s0, funcionando como um contador regressivo de impressões. Paralelamente, \$s1 começa valendo 2 e sofre incrementos de 2 a cada repetição, representando o próximo termo a ser exibido. O loop é encerrado por meio de um beq assim que \$s0 zera. Executando o programa com a entrada de 7 termos, ele gerou corretamente a saída "1 2 4 6 8 10 12". Ao final, o contador regressivo \$s0 atingiu 0x00000000 e a variável do número par (\$s1) parou em 0x0000000e (14), demonstrando que o fluxo de parada atuou no momento exato.

Mapeamento de Registradores

Registrador	Função
\$s0	Contador regressivo do total de termos (N).
\$s1	Guarda os valores pares que são incrementados na sequência.
\$t0	Flag auxiliar para validação inicial de entrada.

Exercício 3: Transcrição e Manipulação de Vetor

Lógica da Solução e Análise de Execução

A transcrição do código em linguagem de alto nível para o Assembly exigiu uma adaptação matemática para o cálculo dos endereços de memória do vetor. Sabendo que cada elemento do tipo `.word` ocupa 4 bytes, o índice das repetições (`$s1`) precisava ser multiplicado por 4 antes de ser somado ao endereço base. Sem a instrução específica de multiplicação, esse alinhamento foi resolvido somando o registrador a ele mesmo sucessivamente. O controle dos laços (equivalentes ao 'for' e 'while') utilizou instruções de verificação e desvio (`slt` e `beq`) para delimitar o limite fixo de 20 interações. A leitura e gravação no vetor ocorreram através de `lw` e `sw`. Nos testes, verificou-se que as posições de memória foram preenchidas e atualizadas dinamicamente, gerando uma progressão aritmética na leitura final correspondente às somas sucessivas do último valor inserido pelo usuário.

Mapeamento de Registradores

Registrador	Função
<code>\$s0</code>	Endereço base do vetor.
<code>\$s1</code>	Índice de iteração dos laços ('i').
<code>\$s2</code>	Limite fixo (20) em dois laços; atua como acumulador ('x') no loop do meio.
<code>\$s3</code>	Total de elementos ('n') usado para alcançar a última posição do vetor.
<code>\$t1, \$t2</code>	Temporários para deslocamento de endereços e cópia de valores da memória.

Exercício 4: Divisão por Subtração

Lógica da Solução e Análise de Execução

O cálculo de divisão baseado em subtrações sucessivas dependeu de um laço contínuo enquanto o dividendo restante (armazenado temporariamente como 'x' em `$t0`) fosse maior ou igual ao divisor (`$s1`). A lógica inversa foi programada usando `slt`: o loop é interrompido via `bne` assim que 'x' se torna menor que o divisor. A cada passagem pelo bloco, o algoritmo subtrai o

divisor e adiciona 1 ao registrador encarregado de contar o quociente (\$s2). Durante a avaliação com entradas robustas, a lógica se provou matemática e computacionalmente exata, processando os ciclos sem falhas até que a condição de parada fosse satisfeita, alocando os valores corretos tanto para o quociente quanto para a sobra da divisão na saída do programa.

Mapeamento de Registradores

Registrador	Função
\$s0, \$s1	Dividendo e divisor informados pelo usuário, respectivamente.
\$s2, \$s3	Guarda o resultado do quociente e o resto da operação.
\$t0	Valor provisório ('x') que sofre as subtrações durante a iteração.
\$t1	Registrador de estado para a avaliação do slt que comanda o laço.

Conclusão

O trabalho desenvolvido demonstrou na prática o comportamento das estruturas de controle de fluxo e manipulação de memória sob a perspectiva da arquitetura MIPS. Superar a ausência de comandos de alto nível (como multiplicadores diretos) empregando lógicas fundamentais com registradores fortaleceu o entendimento do hardware. As avaliações simuladas confirmaram a correção matemática e estrutural de todos os algoritmos construídos.